

SYSTEM DOCUMENTATION & ARCHITECTURE

Progressive Learning Platform

A gamified learning management system featuring sequential mastery unlocking, dynamic assessment pools, and an interactive XP economy.

VERSION

1.0.0 (Production Release)

BACKEND ARCHITECTURE

Node.js • Express 5.x REST API

DATABASE TIER

MySQL 8.x with Connection Pooling

FRONTEND STACK

Vanilla JS (ES6+) • Tailwind CSS

AUTHENTICATION

Stateless JWT + bcryptjs

REPOSITORY

github.com/miradnaan/progressive-learning

1 Executive Summary & Platform Philosophy

Traditional online education platforms often suffer from high student abandonment and superficial content consumption. Learners frequently scrub through instructional videos or jump straight to final exams without truly internalizing prerequisite knowledge.

Progressive Learning solves this core problem by implementing an automated **Mastery-Based Sequential Unlocking Engine**. Instead of self-paced arbitrary navigation, knowledge is structured as an interactive milestone roadmap where each module acts as a strict cognitive prerequisite for the next.



Sequential Gating

Lesson N cannot be accessed until the student demonstrates verifiable mastery by passing Lesson $N-1$'s quiz assessment.



Dynamic XP Economy

Every passed quiz awards 10 XP per correct question, fueling rank progression through Beginner, Intermediate, Advanced, and Expert tiers.



Curriculum Telemetry

Real-time drop-off funnels and KPI analytics empower educators to pinpoint curriculum bottlenecks and optimize lesson retention.

Core Mission

Progressive Learning shifts online education from passive video watching to active retention. By enforcing verifiable mastery at every stage, students build genuine, durable competence.

2 Student Features & Learning Experience

2.1 Interactive Serpentine Learning Roadmap

Inspired by industry leaders in habit-forming education (such as Khan Academy and Duolingo), curriculum progression is visually represented as an organic, winding **S-curve learning trail**:

- **Completed Milestones:** Illustrated with radiant emerald indicators and checkmarks representing mastered modules.
- **Current Active Lesson:** Features animated dual-ring pulsating halos and a subtle floating bob, providing an unmistakable visual beacon.
- **Locked Lessons:** Rendered in muted slate tones with secure lock badges, preventing unauthorized skipping.
- **Dynamic SVG Engine:** Connects all nodes using mathematically calculated cubic bezier tangents ($M_{x1} y1 C_{x1} cpY1, x2 cpY2, x2 y2$) with animated marching dashes that guide the student forward.
- **Course Final Exam Capstone:** A celebratory final milestone badge that unlocks exclusively once all preceding lessons are completed.

2.2 The Quiz & Assessment Engine

Assessment integrity is maintained by decoupling testing from rote memorization:

- **Randomized Question Sampling:** Rather than presenting fixed static tests, the engine queries the question bank and extracts a randomized subset for each attempt.
- **Atomic Scoring & Instant Evaluation:** Answer submissions are graded server-side against protected cryptographic keys, immediately calculating percentage accuracy and awarding earned XP.
- **Mastery Unlocking Trigger:** Upon attaining the passing benchmark, the backend transactionally updates `lesson_progress`, unlocks the subsequent lesson, and triggers a celebratory confetti celebration.

2.3 Gamification, Streaks & Rank Tiers

TIER NAME	XP THRESHOLD	VISUAL BADGE	PRIVILEGES & STATUS
Beginner	0 – 199 XP	Beginner	Foundational tier upon onboarding and first course enrollment.
Intermediate	200 – 499 XP	Intermediate	Demonstrated consistency across multiple completed lessons.
Advanced	500 – 999 XP	Advanced	Proficient student completing full curriculums and final exams.
Expert	1,000+ XP	Expert 🏆	Platform champions with mastery across diverse domains.

Daily Streak Engine: Evaluates login frequency daily. Consecutive logins increment the streak counter, displaying a fire badge in the navigation header. Missing more than 48 hours automatically resets the streak, reinforcing daily discipline.

3 Instructor Suite & Curriculum Authoring

3.1 Executive Analytics & Drop-off Funnel

Instructors are provided with a dedicated administrative dashboard delivering actionable curriculum insights:

- **Telemetry KPIs:** Total enrolled students, overall curriculum completion rate, active course counts, and system-wide quiz average.
- **Per-Lesson Drop-off Funnel:** A visual Chart.js funnel chart plotting completion ratios across each lesson in a sequence. Instructors can instantly identify specific lessons where student completion drops, highlighting ambiguous content or overly difficult quiz questions.
- **Student Engagement Directory:** Detailed roster tracking student progress, individual XP balances, and enrollment dates.

3.2 The 3-Mode Content Studio

Located at `studio.html`, the Content Studio provides a comprehensive visual curriculum authoring suite:

1. Visual Form Mode

Non-technical instructors build structured lessons using clean fields: Introduction, Main Concept, Code Samples, Takeaways, and External Links.

2. Live Preview Mode

Real-time WYSIWYG rendering displaying YouTube video embeds, formatted typography, and prose exactly as a student sees it.

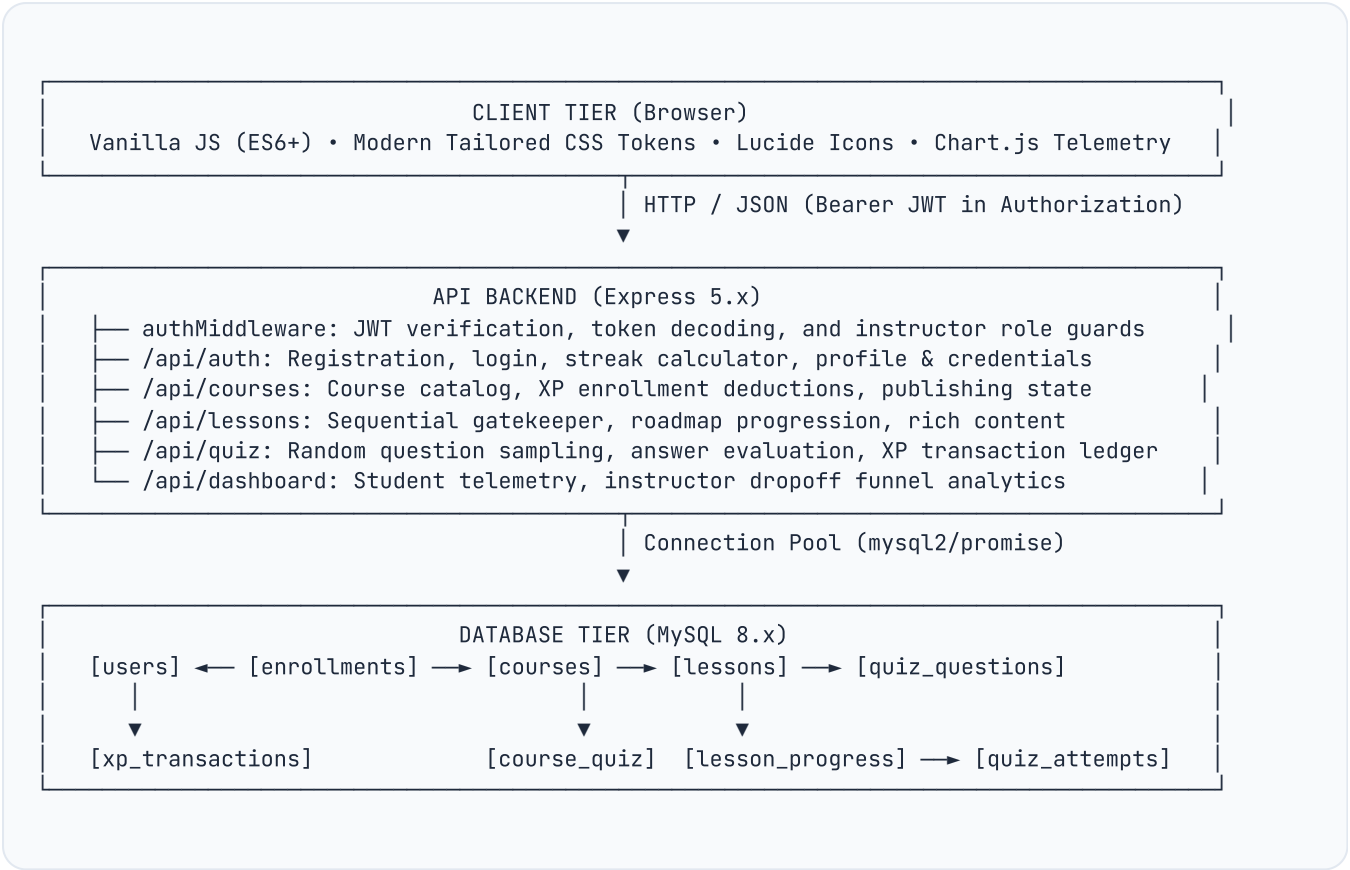
3. Raw HTML Mode

Power-user code editor with direct syntax access for advanced custom styling, iframes, and interactive embeds.

3.3 Course Publishing Lifecycle & Question Bank

Courses support a dual-state lifecycle: **Draft** and **Published**. Draft courses remain completely invisible to students, allowing instructors to build out curriculum, order lessons with reorder buttons (Move Up / Down), and seed question banks before making the course live with a single click.

4 System Architecture & Data Flow



4.1 Technology Stack Rationale

COMPONENT	TECHNOLOGY	ENGINEERING RATIONALE
Runtime	Node.js (v18+)	Asynchronous non-blocking event loop ideal for concurrent API requests and real-time quiz evaluations.
Web Framework	Express 5.x	High-performance, unopinionated routing engine with native modern promise handling.
Relational Database	MySQL 8.x	Strict referential integrity, foreign key cascading, and ACID transactions essential for the XP ledger.
Connection Pool	mysql2/promise	Prepared statement execution preventing SQL injection and connection pooling handling high concurrent loads.
Security	bcryptjs & JWT	Salted password hashing (cost factor 10) and signed stateless token validation without session storage overhead.
Frontend	Vanilla JS + Tailwind	Zero bloated virtual DOM overhead; lightning-fast page transitions and full control over animations.

5 Database Schema Specification

The relational database schema is structured under third normal form (3NF) with enforced foreign key cascades:

5.1 Table: `users`

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
<code>id</code>	INT	PK, AUTO_INCREMENT	Unique primary identifier for the user.
<code>name</code>	VARCHAR(100)	NOT NULL	Full display name of the user.
<code>email</code>	VARCHAR(150)	UNIQUE, NOT NULL	Authentication email address.
<code>password_hash</code>	VARCHAR(255)	NOT NULL	bcrypt salted password hash.
<code>role</code>	ENUM	'student', 'instructor', 'admin'	Access control level (default: 'student').
<code>avatar_url</code>	TEXT	NULL	Custom profile picture image URI.
<code>xp</code>	INT	DEFAULT 0	Cumulative earned experience points balance.
<code>streak</code>	INT	DEFAULT 0	Count of consecutive active study days.
<code>last_active</code>	DATE	NULL	Date of most recent authenticated session.

5.2 Table: **courses**

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	INT	PK, AUTO_INCREMENT	Unique course identifier.
title	VARCHAR(200)	NOT NULL	Course curriculum name.
description	TEXT	NOT NULL	Detailed syllabus and learning goals.
category	VARCHAR(100)	DEFAULT 'General'	Categorization tag for catalog filtering.
duration	VARCHAR(50)	NOT NULL	Estimated time required (e.g. '6 hours').
image_url	TEXT	NULL	Cover thumbnail illustration URL.
xp_cost	INT	DEFAULT 0	XP required to unlock/enroll (0 = Free).
published	TINYINT(1)	DEFAULT 1	1 = Published to students, 0 = Draft.

5.3 Table: **lessons**

COLUMN	TYPE	CONSTRAINTS	DESCRIPTION
id	INT	PK, AUTO_INCREMENT	Unique lesson step identifier.
course_id	INT	FK -> courses.id ON DELETE CASCADE	Associated parent course curriculum.
position	INT	NOT NULL	Sequential step ordering index (1, 2, 3...).
title	VARCHAR(200)	NOT NULL	Lesson topic title.
duration_minutes	INT	DEFAULT 20	Estimated lesson reading/video duration.
video_url	TEXT	NULL	YouTube or hosted video embed link.
content	LONGTEXT	NOT NULL	Full lesson prose, callouts, and code examples.

5.4 Tables: `quiz_questions` & `lesson_progress`

TABLE	KEY COLUMNS	FUNCTION & RELATIONSHIPS
<code>quiz_questions</code>	<code>id</code> , <code>course_id</code> , <code>lesson_id</code> , <code>quiz_type</code> , <code>question_text</code> , <code>options</code> , <code>correct_answer</code>	Stores question text, JSON array of options A-D, and correct key. <code>quiz_type</code> differentiates lesson quizzes from course final exams.
<code>lesson_progress</code>	<code>user_id</code> , <code>lesson_id</code> , <code>passed_quiz</code> , <code>completed_at</code>	Tracks exact user state per lesson. <code>passed_quiz = 1</code> unlocks the next lesson in sequence.
<code>enrollments</code>	<code>user_id</code> , <code>course_id</code> , <code>enrolled_at</code> , <code>is_completed</code>	Maintains user course registration. <code>is_completed = 1</code> is set upon passing the final exam.
<code>xp_transactions</code>	<code>user_id</code> , <code>amount</code> , <code>reason</code> , <code>created_at</code>	Immutable financial ledger auditing all XP credits (quiz rewards, bonus XP) and debits (course enrollments).

6 Comprehensive REST API Specification

6.1 Authentication Endpoints

METHOD	ENDPOINT	AUTH LEVEL	DESCRIPTION
POST	/api/auth/register	Public	Registers student or instructor. Instructors must supply valid <code>invite_code</code> .
POST	/api/auth/login	Public	Validates credentials, updates daily streak, and returns signed JWT token.
GET	/api/auth/me	Authenticated	Fetches authenticated user profile, role, XP balance, and streak count.
PUT	/api/auth/profile	Authenticated	Updates name, avatar image URL, or changes user password with verification.

6.2 Courses & Catalog Endpoints

METHOD	ENDPOINT	AUTH LEVEL	DESCRIPTION
GET	/api/courses	Optional Auth	Returns published catalog with category tags, student counts, and enrollment status.
GET	/api/courses/:id	Optional Auth	Returns course details, syllabus outline, and user unlock state.
POST	/api/courses/:id/enroll	Authenticated	Atomically deducts XP unlock cost and creates enrollment record.
GET	/api/courses/instructor/all	Instructor	Returns all courses including drafts with real-time student counts.
POST	/api/courses	Instructor	Creates new course curriculum with customizable category, duration, and XP cost.
PUT	/api/courses/:id	Instructor	Updates existing course metadata and publishing status.
PATCH	/api/courses/:id/toggle-publish	Instructor	Instant 1-click toggle between Draft and Published status.

METHOD	ENDPOINT	AUTH LEVEL	DESCRIPTION
DELETE	/api/courses/:id	Instructor	Deletes course and cascades all lessons, progress, and questions.

6.3 Lessons & Quizzes Endpoints

METHOD	ENDPOINT	AUTH LEVEL	DESCRIPTION
GET	/api/lessons/course/:courseId	Authenticated	Calculates sequential unlock states and returns structured roadmap nodes.
GET	/api/lessons/:id	Authenticated	Fetches lesson reader content with strict gatekeeper validation.
GET	/api/quiz/lesson/:lessonId	Authenticated	Selects 4 randomized questions from the pool for lesson quiz.
POST	/api/quiz/lesson/:lessonId/submit	Authenticated	Evaluates answers; awards +10 XP per correct answer and unlocks next lesson.
GET	/api/quiz/course/:courseId	Authenticated	Fetches final exam questions (requires all preceding lessons to be completed).
POST	/api/quiz/course/:courseId/submit	Authenticated	Evaluates final exam; marks course complete and awards +100 XP bonus.

7 Security Architecture & Operational Guide



Security Controls

- **Cryptographic Passwords:** All credentials hashed using bcrypt with salt rounds = 10.
- **Stateless JWT Validation:** Protected routes require signed Bearer tokens with 7-day expiration.
- **SQL Injection Immunity:** 100% of database queries execute via parameterized prepared statements.
- **Sequential Gatekeeping:** Server-side validation ensures students cannot access Lesson *N* without passing Lesson *N-1*.



Performance & Scalability

- **Connection Pooling:** Express utilizes `mysql2/promise` pool with automatic connection recycling.
- **Optimized Static Assets:** Frontend assets bundled cleanly with CDN fonts and instant rendering.
- **Database Indexing:** Unique constraints on `users.email` and composite indexes on `(user_id, course_id)`.
- **Atomic XP Transactions:** Balance modifications execute within transactional boundaries.

7.1 Quickstart

```
# 1. Clone repository & install dependencies
git clone https://github.com/miradnaan/progressive-learning.git
cd progressive-learning
npm install

# 2. Configure environment credentials in .env
DB_HOST=127.0.0.1
DB_USER=root
DB_PASS=your_password
DB_NAME=progressive_learning
PORT=3000
JWT_SECRET=super_secure_random_key_here
INSTRUCTOR_INVITE_CODE=PLInstructor2024!SecureKey

# 3. Initialize relational tables & seed demo curriculum
npm run init-db

# 4. Start production server
npm start
```

Testing & Verification Utilities

To reset a test student's progress and verify sequential unlocking from scratch:

```
npm run reset-progress alex@example.com
```